

1 P1

Regularization terms, such as the L_2 regularizer $\|\mathbf{w}\|^2$, help to reduce over-fitting by encouraging weight values to be close to zero. Another way to reduce network complexity is to impose hard constraints on the weights by forming them into groups and requiring that all weights within each group share the same value, in which the shared value is learned from data. This is known as *weight sharing* or *parameter sharing* or *parameter tying*. It means that the number of degrees of freedom is smaller than the number of connections in the network. Usually this is introduced as a way to encode inductive bias into a network to express known invariances. Evaluating the error function gradients for such networks can be done using a small modification to backpropagation although this is handled implicitly through automatic differentiation. We will make extensive use of parameter sharing when we discuss convolutional neural networks. Parameter sharing, however, only applies to particular problems in which the form of the constraints can be specified in advance.

2 P2 Soft weight sharing

Instead of using a hard constraint that forces sets of model parameters to be equal, Nowlan and Hinton (1992) introduced a form of *soft weight sharing* in which a regularization term encourages groups of weights to have similar values. Furthermore, the division of weights into groups, the mean value for each group, and the spread of values within the groups are all determined as part of the learning process.

3 P3

Recall that the simple-weight decay regularizer in (9.1) can be viewed as the negative log of a Gaussian prior distribution over the weights. This encourages all the weights to converge towards a single value of zero. We can instead encourage the weight values to form several groups, rather than just one group, by considering a probability distribution that is a *mixture* of Gaussians. The means $\{\mu_j\}$ and variances $\{\sigma_j^2\}$ of the Gaussian components, as well as the mixing coefficients $\{\pi_j\}$, will be considered as adjustable parameters to be determined as part of the learning process.

Thus, we have a probability density of the form

$$p(\mathbf{w}) = \prod_i \left\{ \sum_{j=1}^K \pi_j \mathcal{N}(w_i \mid \mu_j, \sigma_j^2) \right\} \quad (9.21)$$

where K is the number of components in the mixture. Taking the negative logarithm then leads to a regularization function of the form

$$\Omega(\mathbf{w}) = - \sum_i \ln \left(\sum_{j=1}^K \pi_j \mathcal{N}(w_i \mid \mu_j, \sigma_j^2) \right). \quad (9.22)$$

The total error function is then given by

$$\tilde{E}(\mathbf{w}) = E(\mathbf{w}) + \lambda \Omega(\mathbf{w}) \quad (9.23)$$

where λ is the regularization coefficient.

4 P4

This error is minimized jointly with respect to the weights $\{w_i\}$ and with respect to the parameters $\{\pi_j, \mu_j, \sigma_j^2\}$ of the mixture model. This can be done using gradient descent, which requires that we evaluate the derivatives of $\Omega(\mathbf{w})$ with respect to all the learnable parameters. To do this, it is convenient to regard the $\{\pi_j\}$ as *prior* probabilities for each component to have generated a weight value, and to introduce the corresponding posterior probabilities, which are given by Bayes' theorem:

$$\gamma_j(w_i) = \frac{\pi_j \mathcal{N}(w_i \mid \mu_j, \sigma_j^2)}{\sum_k \pi_k \mathcal{N}(w_i \mid \mu_k, \sigma_k^2)}. \quad (9.24)$$

The derivatives of the total error function with respect to the weights are then given by

$$\frac{\partial \tilde{E}}{\partial w_i} = \frac{\partial E}{\partial w_i} + \lambda \sum_j \gamma_j(w_i) \frac{(w_i - \mu_j)}{\sigma_j^2}. \quad (9.25)$$

The effect of the regularization term is therefore to pull each weight towards the centre of the j th Gaussian, with a force proportional to the posterior probability of that Gaussian for the given weight. This is precisely the kind of effect that we are seeking.

5 P5

Derivatives of the error with respect to the centres of the Gaussians are also easily computed to give

$$\frac{\partial \tilde{E}}{\partial \mu_j} = \lambda \sum_i \gamma_j(w_i) \frac{(\mu_j - w_i)}{\sigma_j^2} \quad (9.26)$$

which has a simple intuitive interpretation, because it pushes μ_j towards an average of the weight values, weighted by the posterior probabilities that the respective weight parameters were generated by component j .

6 P6

To ensure that the variances $\{\sigma_j^2\}$ remain positive, we introduce new variables $\{\xi_j\}$ defined by

$$\sigma_j^2 = \exp(\xi_j) \quad (9.27)$$

and an unconstrained minimization is performed with respect to the $\{\xi_j\}$. The associated derivatives are then given by

$$\frac{\partial \tilde{E}}{\partial \xi_j} = \frac{\lambda}{2} \sum_i \gamma_j(w_i) \left(1 - \frac{(w_i - \mu_j)^2}{\sigma_j^2} \right). \quad (9.28)$$

This process drives σ_j towards a weighted average of the squared deviations of the weights around the corresponding centre μ_j , where the weighting coefficients are again given by the posterior probability

that each weight is generated by component j .

7 P7

For the derivatives with respect to the mixing coefficients π_j , we need to take account of the constraints

$$\sum_j \pi_j = 1, \quad 0 \leq \pi_i \leq 1, \quad (9.29)$$

which follow from the interpretation of the π_j as prior probabilities. This can be done by expressing the mixing coefficients in terms of a set of auxiliary variables $\{\eta_j\}$ using the *softmax* function given by

$$\pi_j = \frac{\exp(\eta_j)}{\sum_{k=1}^K \exp(\eta_k)}. \quad (9.30)$$

The derivatives of the regularized error function with respect to the $\{\eta_j\}$ then take the form

$$\frac{\partial \tilde{E}}{\partial \eta_j} = \lambda \sum_i (\pi_j - \gamma_j(w_i)). \quad (9.31)$$

We see that π_j is therefore driven towards the average posterior probability for mixture component j .

8 P8

A different application of soft weight sharing (Lasserre, Bishop, and Minka, 2006) introduces a principled approach that combines the unsupervised training of a generative model with the supervised training of a corresponding discriminative model. This is useful in situations where we have a significant amount of unlabelled data but where labelled data is in short supply. The generative model has the advantage that all of the data can be used to determine its parameters, whereas only the labelled examples directly inform the parameters of the discriminative model. However, a discriminative model can achieve better generalization when there is model mis-specification, in other words when the model does not exactly describe the true distribution that generates the data, as is typically the case. By introducing a soft tying of the parameters of the two models, we obtain a well-defined hybrid of gener-

ative and discriminative approaches that can be robust to model mis-specification while also benefiting from being trained on unlabelled data.